LEADERS UNIVERSITY / LICENCE BI

Démarrer : Windows/WSL ou Ubuntu, Codex Free, Git et GitHub

Solen BI · 22 h · votre poste, vos comptes et votre premier cycle Git à deux

22 h	2	0
5 × 4 H + SOUTENANCE	PAR BINÔME, 2 COMPTES	LIGNE DE CODE À LA MAIN

SOLEN DIGITAL · MAISON ZITOUNA (FICTIFS)

Poste → Codex → Git → GitHub → première PR à deux

01 / DÉMARRER

Youssef Douma · Leaders University

Édition pédagogique · 28 octobre 2026 · données fictives

01 / DÉMARRER · SOMMAIRE

Sommaire et liste de départ

Ce livret vous mène du poste vide au premier cycle Git réussi à deux, en deux temps. **Parties A à C : séance 0, chez vous, avant la séance 1** (≈ 2 h, estimation, pas une mesure). Le professeur envoie ce livret une semaine avant la séance 1 et répond aux blocages par message : copiez la commande et la dernière ligne affichée. **La séance 1 commence par 30 min de vérification** de la liste « Fin de la séance 0 » (page 18) ; les parties D et E se font en séance, à deux. Les cadres « Capture prévue » marquent des écrans pas encore photographiés : le professeur les montre en séance.

Partie	Quand · pour qui	Page
A · Comprendre son poste, lire un terminal, quatre mots	Chez vous · tous	3
B · Installer : route Windows (WSL2)	Chez vous · Windows	6
B · Installer : route Ubuntu natif	Chez vous · Ubuntu	10
C · Codex, compte GitHub, identité Git ; fin de la séance 0	Chez vous · tous	12
D · GitHub à deux : dépôt, invitation, venv, protection	Séance 1 · le binôme	19
E · Premier cycle à deux : la PR n°1	Séance 1 · le binôme	25
Annexes : Power BI Desktop, Codex CLI, récapitulatif	Tous	31

Avant de commencer, chacun coche

À préparer	Détail
[ ] Une adresse e-mail personnelle	Pour GitHub et ChatGPT ; pas d'adresse partagée.
[ ] Un compte ChatGPT Free	Aucun paiement, aucune clé API.
[ ] Un poste pris en charge	Windows 10 version 2004+ ou Windows 11 ; ou Ubuntu 24.04 / 26.04 LTS. Quelques Go libres, un réseau stable.
[ ] Un accès Windows en séance 5	Power BI Desktop ne s'installe pas sur Ubuntu.
[ ] Les rôles du binôme	**A** crée le dépôt GitHub et invite **B**. Ensuite, auteur et relecteur alternent à chaque pull request.

01 / DÉMARRER · A · COMPRENDRE SON POSTE · CHEZ VOUS

Mon poste : qui fait quoi

Lundi matin, chez Solen Digital, vous recevez un ordinateur et une mission : industrialiser la BI de Maison Zitouna. Avant la première ligne de données, il faut savoir quel outil fait quoi. Six mots suffisent.

Outil	Ce que c'est	Exemple concret
Windows	Le système de votre PC : fenêtres, Explorateur, Microsoft Store.	Vous installez l'application ChatGPT et Power BI Desktop depuis le Store.
PowerShell	La ligne de commande de Windows. On tape une commande, Windows répond.	`wsl --install` installe Ubuntu. On ne s'en sert presque que pour cela.
Ubuntu (dans WSL)	Un vrai Linux qui tourne dans Windows grâce à WSL2. Il a ses propres dossiers et son terminal.	Python, Git et le dossier `~/solen-bi` vivent ici, pas sur `C:`.
Application ChatGPT (Codex)	L'application de bureau où vous parlez à Codex. Codex lit vos fichiers, propose un changement, lance des commandes.	« Lis README.md et dis-moi ce que fait ce dépôt. Ne modifie rien. »
GitHub	Le site qui garde la copie partagée du dépôt du binôme, les pull requests, les revues et la CI.	A propose un changement, B le relit et l'approuve sur github.com.
Power BI Desktop	Le logiciel gratuit de Microsoft pour modèles et rapports. Windows uniquement.	Séance 5 : vous chargez les exports, créez les relations et une mesure DAX.



Ubuntu natif ?

Si votre PC est sous Ubuntu 24.04 ou 26.04, il n'y a ni WSL ni PowerShell : tout se passe dans le Terminal Ubuntu. Power BI Desktop exige un poste Windows : prévoyez-en un (salle de cours, binôme) pour la séance 5.

01 / DÉMARRER · A · COMPRENDRE SON POSTE · CHEZ VOUS

Lire un terminal et les blocs du livret

Un terminal est une fenêtre de texte. Vous tapez une **commande**, Entrée, le système affiche une **sortie** puis redonne la main avec l'**invite** (par exemple `amira@pc:~$`). ~ est votre dossier personnel Linux, `/home/<utilisateur-linux>`.

Terminal Ubuntu (bash)

```
pwd
```

Sortie attendue · ne pas recopier

```
/home/amira
```

Chaque bloc sombre dit où l'utiliser : **PowerShell (Windows)**, **Terminal Ubuntu (bash)**, **Demande à Codex** ou **Sortie attendue**. Une sortie se lit et se compare ; elle ne se tape jamais, l'invite non plus.

Repère	Ce que cela veut dire
`<chevrons>`	À remplacer en entier, chevrons compris : `<compteA>` devient par exemple `amira-bk`.
`sudo`	Exécute en administrateur. Ubuntu demande votre mot de passe Linux : il ne s'affiche pas quand vous tapez. C'est normal.
Coller	Terminal Ubuntu : clic droit, ou Ctrl + Maj + V. Ctrl + C y **interrompt** une commande en cours.
Ligne finie par \	La commande continue à la ligne suivante : copiez tout le bloc d'un coup.
Erreur	Lisez la **dernière** ligne d'abord : elle nomme souvent le fichier ou la commande en cause.

Deux règles de prudence

Ne lancez jamais une commande que vous ne comprenez pas, même proposée par Codex : « Explique cette commande ligne par ligne, sans l'exécuter. » Vos mots de passe ne s'écrivent nulle part : ni dans le dépôt, ni dans une demande à Codex.

01 / DÉMARRER · A · COMPRENDRE SON POSTE · CHEZ VOUS

Quatre mots avant d'installer : chemin, éditeur, encodage, JSON

Chemin

Un **chemin** dit où est un fichier. `/` est la racine de Linux : un chemin qui commence par `/` est **absolu** (`/home/amira/cours-bi`). `~` abrège votre dossier personnel : `~/cours-bi` = `/home/amira/cours-bi`. Sans `/` au début, le chemin part du dossier courant, celui qu'affiche `pwd` (`tronc-commun/sources/lot1/ventes.csv`). `..` est le dossier parent, `.` le dossier courant. Exemple, une fois le cours cloné : `cd ~/cours-bi/pdf` puis `pwd` affiche `/home/amira/cours-bi/pdf` ; `cd ..` puis `pwd` affiche `/home/amira/cours-bi`.

Éditeur

Un **éditeur** ouvre un fichier texte pour le lire ou le modifier. Pour **lire** sans risque : `cat` (tout), `head -5` (5 premières lignes), `less` (page par page, q pour quitter). Pour **écrire** un texte (journal, README) : l'éditeur de l'application Codex si votre version en propose un, ou `nano` dans le terminal : `nano notes/JOURNAL.md`, puis **Ctrl + O** et Entrée pour enregistrer, **Ctrl + X** pour quitter. Git ouvre aussi nano pour un message de fusion (réglage de GitHub 1).

Encodage

Un fichier texte est une suite d'**octets** ; l'**encodage** est la table qui dit quel caractère chaque octet représente. Le cours écrit tout en **UTF-8** ; un vieux Windows ou Excel écrit parfois en **cp1252**. Le mot « août » : en UTF-8, « û » prend 2 octets (`C3 BB`) ; en cp1252, un seul (`FB`). Un fichier UTF-8 lu comme du cp1252 affiche `aoÃ»t` ; l'inverse affiche un losange ou un point d'interrogation à la place du « û ». Devant ces signes, le fichier est intact : c'est la lecture qui se trompe de table.

JSON

JSON range des fiches dans un fichier texte : entre accolades `{ }`, des paires `"clé": valeur` ; entre crochets `[ ]`, une liste. Une valeur peut être elle-même une liste de fiches. Extrait de `boutiques.json` du tronc commun :

Fichier tronc-commun/sources/lot1/boutiques.json · extrait

```
{
  "boutiques": [
    { "code": "NBE",
      "nom": "Maison Zitouna Nabeul",
      "historique": [
        { "zone": "Cap Bon", "valide_depuis": "2021-05-10" }
      ] }
  ]
}
```

Lecture : une liste `boutiques` ; chaque boutique est une fiche (`code`, `nom`…) qui contient une liste `historique` de versions de zone. Au lot 2, NBE aura deux versions.

01 / DÉMARRER · B · ROUTE WINDOWS · CHEZ VOUS

Windows 1 · Ubuntu, Python et Git

WSL (Windows Subsystem for Linux) fait tourner Ubuntu dans Windows. Prérequis Microsoft : Windows 10 version 2004 ou plus récent, ou Windows 11. Menu Démarrer → **PowerShell** → clic droit → **Exécuter en tant qu'administrateur**, puis :

PowerShell (Windows)

```
wsl --install
```

Attendez la fin et **redémarrez le PC**. Contrôlez ensuite dans un PowerShell normal :

PowerShell (Windows)

```
wsl -l -v
```

Sortie attendue · ne pas recopier

```
  NAME      STATE           VERSION
* Ubuntu    Running         2
```

Réussite : Ubuntu en VERSION 2 (STATE peut valoir Stopped)

Texte d'aide de wsl au lieu d'une installation : `wsl --install -d Ubuntu`. Blocage à 0.0 % : `wsl --install --web-download -d Ubuntu`. VERSION 1 : `wsl --set-version Ubuntu 2`. Message sur la virtualisation : appelez le professeur, ne touchez pas seul au BIOS/UEFI.

Premier utilisateur Linux et mise à jour

Ouvrez **Ubuntu** (menu Démarrer). Il demande un nom et un mot de passe **Linux**, sans lien avec Windows : nom en minuscules sans espace (`amira`) ; le mot de passe ne s'affiche pas. Puis mettez Ubuntu à jour :

Terminal Ubuntu (bash)

```
sudo apt update && sudo apt upgrade -y
```

Ligne finale en `E:` : erreur, souvent le réseau, relancez. `Sorry, try again.` : mot de passe mal tapé. Oublié : PowerShell, `wsl -u root`, `passwd <utilisateur-linux>`, `exit`.

Python, venv et Git

Python exécute le pipeline écrit par Codex ; **venv** est une boîte à outils Python propre au projet (pandas, pytest…) ; **Git** garde l'historique des fichiers.

01 / DÉMARRER · B · ROUTE WINDOWS · CHEZ VOUS

Windows 1 · Ubuntu, Python et Git · suite

Terminal Ubuntu (bash)

```
sudo apt install -y python3 python3-venv python3-pip git sqlite3
python3 --version
git --version
```

Sortie attendue · ne pas recopier

```
Python 3.12.3
git version 2.43.0
```

Les numéros dépendent de votre Ubuntu : 24.04 fournit Python 3.12, 26.04 Python 3.14. Le cours demande **3.11 ou plus**. Le venv sera créé plus tard, dans le dépôt du binôme.

`Unable to locate package` : `sudo apt update` d'abord. `externally-managed-environment` : normal hors venv. `python` introuvable : tapez `python3`.

Où vit le projet

Vos deux dossiers vivent **dans Linux** : `~/cours-bi` (le cours) et `~/solen-bi` (le dépôt du binôme), créés plus loin par `git clone`. Microsoft recommande le système de fichiers WSL pour les outils Linux. Évitez `C:\Users\...` (`/mnt/c/...`) : lent et source d'erreurs.

Le même dossier vu depuis…	Chemin
Ubuntu (terminal, agent WSL)	`~/solen-bi` = `/home/<utilisateur-linux>/solen-bi`
Windows (Explorateur, sélecteur)	`\\wsl$\Ubuntu\home\<utilisateur-linux>\solen-bi`

Voir un dossier Linux dans l'Explorateur : `explorer.exe .` (le point = dossier courant). `\\wsl$\` et `\\wsl.localhost\` désignent le même endroit.

01 / DÉMARRER · B · ROUTE WINDOWS · CHEZ VOUS

Windows 2 · L'application ChatGPT et l'agent WSL

Installez l'application de bureau ChatGPT pour Windows depuis le Microsoft Store (lien sur la page officielle « Desktop app for Windows »), ouvrez-la et connectez-vous avec **votre** compte ChatGPT Free. La doc donne aussi une commande :

PowerShell (Windows)

```
winget install --id 9PLM9XGG6VKS -s msstore
```

Par défaut, l'agent Codex travaille côté Windows. Pour qu'il travaille dans Ubuntu, là où sont Python, Git et vos dossiers :

- Ouvrez **Settings** (Ctrl + ,).
- **Agent environment** → choisissez **Windows Subsystem for Linux (WSL)**.
- **Integrated terminal shell** → choisissez **WSL** : le terminal intégré (Ctrl + `) ouvrira Ubuntu.
- **Quittez l'application** (Ctrl + Q) puis rouvrez-la : le changement d'agent ne prend effet qu'après redémarrage.

CAPTURE À RÉALISER · codex-app-reglages.png

Capture prévue, non encore réalisée : codex-app-reglages.png. Capture prévue · Windows 11, application ChatGPT, Settings. Zone : Agent environment et Integrated terminal shell. Action : choisir WSL dans les deux. Réussite : après redémarrage, le terminal intégré affiche une invite Ubuntu.

Si l'application signale que Git manque

La doc Windows indique que sans Git installé côté Windows, l'application ne peut pas utiliser certaines fonctions. Installez alors Git for Windows (git-scm.com) ; il sert aussi de route de secours pour l'authentification.

01 / DÉMARRER · B · ROUTE WINDOWS · CHEZ VOUS

Windows 3 · Récupérer le cours et l'ouvrir dans Codex

Le dépôt du cours est public : on le **clone** (copie complète avec historique) sans compte. Adresse : page GitHub du cours, bouton **Code** → **HTTPS** → copier.

Terminal Ubuntu (bash)

```
git clone https://github.com/jodouma/leaders-bi-ia-etl-28h ~/cours-bi
ls ~/cours-bi
```

Sortie attendue · ne pas recopier

```
Cloning into '/home/amira/cours-bi'...
AGENTS.md  README.md  aide-memoire  docs  modele-binome  pdf  prompts ...
```

Dans l'application, mode **Codex** : **Add new project** ou Ctrl + O. Dans l'Explorateur qui s'ouvre, tapez `\\wsl$\` dans la barre d'adresse, puis **Ubuntu** → **home** → `<utilisateur-linux>` → **cours-bi**.

Demande à Codex · texte à adapter

```
Je débute. Lis README.md et AGENTS.md du dossier ouvert.
Explique en 5 phrases à quoi sert ce dépôt et où sera mon travail.
Ne modifie aucun fichier et ne lance aucune commande.
```

Comparez la réponse aux fichiers réels : Codex doit citer `modele-binome` et `AGENTS.md`. S'il invente un fichier, dites-le-lui : c'est déjà de la revue.

Erreurs courantes

`\\wsl$\` vide : ouvrez une fenêtre Ubuntu, rafraîchissez. Codex ne trouve pas `python3` ou `git` : l'agent est encore Windows natif, refaites Windows 2. `destination path ... already exists` : le cours est déjà là, `cd ~/cours-bi` puis `git pull`.

À ce stade, je sais…

…dire si je suis dans PowerShell ou Ubuntu en lisant l'invite ; montrer Ubuntu en VERSION 2, `python3` 3.11+ et `git` ; retrouver `~/cours-bi` depuis `\\wsl$\` ; montrer l'agent et le terminal sur WSL ; obtenir une réponse de Codex sans modification. **Contrôle** : rouvrez l'app, terminal intégré (Ctrl + accent grave), `pwd` doit afficher un chemin `/home/...`.

01 / DÉMARRER · B · ROUTE UBUNTU NATIF · CHEZ VOUS

Ubuntu natif 1 · Mise à jour, Python et Git

Sur un PC Ubuntu, tout se passe dans le **Terminal** (Ctrl + Alt + T). Vérifiez d'abord la version : l'application ChatGPT pour Linux (en preview) prend en charge Ubuntu **24.04 LTS** et **26.04 LTS**.

Terminal Ubuntu (bash)

```
lsb_release -ds
uname -m
```

Sortie attendue · ne pas recopier

```
Ubuntu 24.04.3 LTS
x86_64
```

`x86_64` signifie processeur x64 (paquet `amd64`) ; `aarch64` signifie ARM64 (paquet `arm64`). Mettez à jour, puis installez Python, venv, pip et Git :

Terminal Ubuntu (bash)

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y python3 python3-venv python3-pip git sqlite3
python3 --version
git --version
```

Sortie attendue · ne pas recopier

```
Python 3.12.3
git version 2.43.0
```

Ubuntu 24.04 fournit Python 3.12, Ubuntu 26.04 Python 3.14 : les deux conviennent (3.11+ demandé). Vos dossiers seront `~/cours-bi` et `~/solen-bi`, créés par `git clone`.

Erreurs courantes

`Unable to locate package` : `sudo apt update` d'abord. `Sorry, try again.` : mot de passe mal tapé (il ne s'affiche pas). `externally-managed-environment` : normal hors venv, n'installez des bibliothèques que dans le venv du projet.

01 / DÉMARRER · B · ROUTE UBUNTU NATIF · CHEZ VOUS

Ubuntu natif 2 · L'application ChatGPT pour Linux

Téléchargez le paquet `.deb` depuis la page officielle « Desktop app for Linux » : `chatgpt_amd64.deb` pour x64, `chatgpt_arm64.deb` pour ARM64. Sur un Ubuntu en français, le dossier de téléchargement s'appelle souvent `~/Téléchargements` (la doc écrit `~/Downloads`).

Terminal Ubuntu (bash)

```
cd ~/Téléchargements
sudo apt install ./chatgpt_amd64.deb
```

Le `./` est obligatoire : il dit à apt d'installer **ce fichier**. Lancez l'application depuis le menu, ou :

Terminal Ubuntu (bash)

```
chatgpt
```

Connectez-vous avec votre compte ChatGPT Free. Pas de réglage WSL ici : l'agent travaille directement dans votre Ubuntu.

Récupérer le cours et l'ouvrir

Terminal Ubuntu (bash)

```
git clone https://github.com/jodouma/leaders-bi-ia-etl-28h ~/cours-bi
ls ~/cours-bi
```

Adresse : page GitHub du cours, **Code** → **HTTPS** → copier. Dans l'application, mode **Codex** : **Add new project** (ou Ctrl + O) → dossier personnel → **cours-bi**. Posez la demande en lecture seule de la page Windows 3 et comparez la réponse aux fichiers.

Erreurs courantes

`E: Unsupported file ./chatgpt_amd64.deb` : vous n'êtes pas dans le dossier du téléchargement, ou le nom diffère (`ls` pour le voir). Paquet ARM64 sur PC x64 : reprenez le bon fichier. Ubuntu 22.04 ou plus ancien : non pris en charge par la preview, voyez le professeur.

À ce stade, je sais…

…donner ma version d'Ubuntu et mon processeur ; afficher `python3` 3.11+ et `git` ; lancer l'application avec `chatgpt` ; ouvrir `~/cours-bi` dans Codex et obtenir une réponse sans modification.

01 / DÉMARRER · C · VOS COMPTES ET RÉGLAGES · CHEZ VOUS

Codex 1 · Compte Free, modèle et effort

Dans l'application, choisissez le mode **Codex** (Alt + 3 bascule entre Chat, Work et Codex), puis **New chat** dans votre projet. Trois réglages comptent.

Réglage	Où	Pour ce cours
Modèle	Sélecteur de modèle (Ctrl + Maj + M) ou `/model`	**GPT-6 Luna**, le modèle annoncé pour Free, quand il est disponible.
Effort de raisonnement	Même sélecteur, préréglage « Luna High » ou **Advanced** : Light, Medium, High, Extra High	**High** pour commencer (« Start with High for Luna »). Light pour une question simple.
Vitesse	Standard pour Free	Standard est une **vitesse**, pas un effort. Ne cherchez pas d'autre option.

CAPTURE À RÉALISER · codex-app-modele-effort.png

Capture prévue, non encore réalisée : codex-app-modele-effort.png. Capture prévue · application ChatGPT (bureau), mode Codex, compte Free. Zone : sélecteur de modèle sous la zone de saisie. Action : choisir GPT-6 Luna puis High. Réussite : le composer affiche Luna et High.

Voir son usage

- `/status` dans le composer : identifiant du chat, **contexte** utilisé et **limites** d'usage.
- `/usage` : ouvre les réglages d'usage, si disponibles avec votre connexion ChatGPT. Tableau de bord web : chatgpt.com/codex/settings/usage.
- Si un réglage de ce livret n'apparaît pas sur **votre** compte, n'en inventez pas un autre : notez ce que vous voyez et montrez-le au professeur.

01 / DÉMARRER · C · VOS COMPTES ET RÉGLAGES · CHEZ VOUS

Codex 2 · Une demande, un diff, un contrôle

Vous n'écrivez pas le code : vous écrivez la **demande**, vous lisez le **diff** (les lignes ajoutées en vert, retirées en rouge), vous lancez le **contrôle** et vous décidez. Une bonne demande a trois parties.

1. Un petit objectif
2. Les fichiers concernés
3. Le contrôle attendu
4. Lire le diff, puis décider

Demande à Codex · texte à adapter

```
Objectif : dans notes/JOURNAL.md, entrée PR #1, remplace la date,
<prénom A> et <prénom B> par : 2026-11-02, Amira, Sami.
Fichier : notes/JOURNAL.md seulement.
Contrôle : git diff ne montre que ces 2 lignes.
Propose d'abord ton plan en 3 étapes, attends mon accord.
```

Vous ferez cette demande pour de vrai au Cycle 1, dans `~/solen-bi`, avec vos prénoms et la date du jour.

- `/plan` active le mode plan : Codex propose les étapes avant d'agir.
- L'onglet de revue (Ctrl + Maj + G) montre les fichiers modifiés et leur diff.
- `/review` lance une revue des changements non commités ou une comparaison avec une branche de base.
- Une commande annoncée par Codex n'est pas une preuve : lancez-la vous-même et lisez la sortie.

CAPTURE À RÉALISER · codex-app-diff.png

Capture prévue, non encore réalisée : codex-app-diff.png. Capture prévue · application ChatGPT (bureau), mode Codex, dépôt du binôme. Zone : onglet de revue. Action : lire les lignes changées dans notes/JOURNAL.md. Réussite : un seul fichier modifié, deux lignes changées.

01 / DÉMARRER · C · VOS COMPTES ET RÉGLAGES · CHEZ VOUS

Compte Free : ce que dit la doc, ce que vous vérifiez

Documentation OpenAI lue le **28 octobre 2026**. Elle peut changer ; votre écran fait foi pour votre compte. Ce livret ne promet aucun quota.

La doc dit	Vous vérifiez vous-même
Free : « GPT-6 Luna at Standard speed in the desktop app, subject to rollout ».	Luna apparaît-il dans le sélecteur ? « Subject to rollout » : l'accès peut arriver plus tard sur certains comptes.
La page des tarifs ne cite pour Free que l'application de bureau ; le CLI, l'IDE et le web apparaissent à partir de Plus.	Le cours n'exige que l'application. Le CLI reste une annexe facultative.
Le nombre de messages dépend du modèle, de la taille et de la complexité des tâches ; des limites hebdomadaires peuvent s'appliquer.	`/status` avant une grosse demande ; notez l'heure de reprise si la limite est atteinte.
Les efforts visibles dans l'app : Light, Medium, High, Extra High.	Lesquels votre compte affiche. Commencez par High.

Tokens, contexte, effort, quota : quatre mots différents

- **Token** : un petit morceau de texte. Vos fichiers, la demande et la réponse se comptent en tokens.
- **Contexte** : ce que Codex garde en tête dans ce chat. Un chat trop long se remplit ; ouvrez un nouveau chat avec un résumé court.
- **Effort** : combien Codex réfléchit avant de répondre. Plus d'effort demande souvent plus d'usage par demande ; mais un effort trop faible peut obliger à reprendre la demande plusieurs fois.
- **Quota** : votre allocation d'usage sur une période. Elle varie, elle se consulte, elle ne se devine pas.

Quota atteint ?

Gardez votre travail (commit sur votre branche), notez l'heure de reprise affichée, continuez ce qui ne demande pas Codex : lire le diff, lancer les contrôles, écrire le journal, préparer la revue. Aucun achat ni clé API n'est demandé.

01 / DÉMARRER · C · VOS COMPTES ET RÉGLAGES · CHEZ VOUS

GitHub 1 · Compte et identité Git

Chacun crée **son** compte sur https://github.com/signup et suit les étapes, dont la vérification de l'adresse e-mail. Le plan gratuit suffit.

- Nom d'utilisateur sobre et durable (il apparaît dans l'historique), par exemple `amira-bk`. Envoyez-le à votre binôme, rien d'autre.
- Activez l'authentification à deux facteurs (2FA), recommandée par GitHub et Microsoft.
- **Settings → Emails** → cochez **Keep my email addresses private** et notez l'adresse …`@users.noreply.github.com` affichée.

Se présenter à Git

Chaque commit (une photo datée de vos fichiers, voir partie D) porte un nom et une adresse : réglez-les une fois par poste, avec l'adresse noreply. Trois réglages de plus : la branche par défaut s'appelle `main` ; `git pull` fait une **fusion simple** quand votre copie et GitHub ont avancé chacune de leur côté ; l'éditeur de Git est nano.

Terminal Ubuntu (bash)

```
git config --global user.name "<Prénom Nom>"
git config --global user.email "<id+compte>@users.noreply.github.com"
git config --global init.defaultBranch main
git config --global pull.rebase false
git config --global core.editor nano
git config --global --list
```

Sortie attendue · ne pas recopier

```
user.name=Amira Ben Khalifa
user.email=12345678+amira-bk@users.noreply.github.com
init.defaultbranch=main
pull.rebase=false
core.editor=nano
```

Avec `pull.rebase false`, si votre branche et celle de GitHub ont toutes deux avancé, `git pull` les fusionne et ouvre nano avec un message déjà écrit (« Merge branch… ») : gardez-le, **Ctrl + O**, Entrée, **Ctrl + X**. Si les deux changements touchent les mêmes lignes, Git annonce un conflit : page « Résoudre un conflit ».

À ce stade, je sais…

…me connecter à github.com, donner mon nom d'utilisateur, et montrer `git config --global --list` avec mon nom et mon adresse noreply.

01 / DÉMARRER · C · VOS COMPTES ET RÉGLAGES · CHEZ VOUS

GitHub 2 · Relier Ubuntu à GitHub

Pour pousser vers GitHub, Git doit prouver qui vous êtes. En HTTPS, GitHub recommande **GitHub CLI** ou **Git Credential Manager**. Route principale du cours : **GitHub CLI** (gh), identique sous WSL et Ubuntu natif, installée depuis le dépôt officiel. Copiez **tout** le bloc d'un coup.

Terminal Ubuntu (bash)

```
sudo apt install -y wget
sudo mkdir -p -m 755 /etc/apt/keyrings
KEY=/etc/apt/keyrings/githubcli-archive-keyring.gpg
wget -qO- https://cli.github.com/packages/githubcli-archive-keyring.gpg \
  | sudo tee "$KEY" > /dev/null
sudo chmod go+r "$KEY"
ARCH=$(dpkg --print-architecture)
SRC="https://cli.github.com/packages stable main"
echo "deb [arch=$ARCH signed-by=$KEY] $SRC" \
  | sudo tee /etc/apt/sources.list.d/github-cli.list > /dev/null
sudo apt update
sudo apt install -y gh
gh --version
```

Sortie attendue · ne pas recopier

```
gh version <numéro> (<date>)
```

Pourquoi pas `sudo apt install gh` tout court ?

Le paquet communautaire d'Ubuntu est signalé comme cassé par les mainteneurs de GitHub CLI (versions 2.45/2.46). Le bloc suit leurs instructions officielles, découpées en lignes courtes.

Se connecter avec le navigateur

Terminal Ubuntu (bash)

```
gh auth login
```

Répondez avec les flèches et Entrée (libellés pouvant varier selon la version) :

01 / DÉMARRER · C · VOS COMPTES ET RÉGLAGES · CHEZ VOUS

GitHub 2 · Relier Ubuntu à GitHub · suite

Question (en anglais)	Votre réponse
Where do you use GitHub?	GitHub.com
Preferred protocol for Git operations?	HTTPS
Authenticate Git with your GitHub credentials?	Y (oui)
How would you like to authenticate?	Login with a web browser

Le terminal affiche un **code à usage unique**. Ouvrez https://github.com/login/device dans votre navigateur, connectez-vous, tapez le code, autorisez. Sous WSL, si le navigateur ne s'ouvre pas seul, ouvrez l'adresse vous-même. Puis :

Terminal Ubuntu (bash)

```
gh auth setup-git
gh auth status
```

Sortie attendue · ne pas recopier

```
github.com
  ✓ Logged in to github.com account <votre-compte> (...)
  - Git operations protocol: https
```

Route de secours : Git Credential Manager

Si gh ne s'installe pas : sous Windows, installez **Git for Windows** (il inclut Git Credential Manager) ; dans Ubuntu WSL, `git credential-manager --version` doit alors répondre, et une fenêtre GitHub s'ouvre au premier `git push`. Sur Ubuntu natif, appelez le professeur.

Jamais de jeton dans le dépôt

Aucun mot de passe, jeton ou clé dans un fichier, une URL `git remote` ou une demande à Codex. « Password authentication is not supported » : refaites `gh auth login` puis `gh auth setup-git`.

01 / DÉMARRER · C · VOS COMPTES ET RÉGLAGES · CHEZ VOUS

Fin de la séance 0 · la liste vérifiée en début de séance 1

La séance 1 commence par **30 min de vérification des postes** : vous **montrez** chaque ligne au professeur ou à votre binôme. Préparez-la chez vous. Ce qui ne marche pas, signalez-le **avant** la séance, par message, avec la commande et la dernière ligne affichée.

[ ] Je montre…	Commande ou écran	Réussite
[ ] Ubuntu prêt	`wsl -l -v` (PowerShell) ; Ubuntu natif : `lsb_release -ds`	VERSION 2 ; ou 24.04 / 26.04 LTS
[ ] Python, Git et sqlite3	`python3 --version`, `git --version`, `sqlite3 --version`	Python 3.11 ou plus ; deux autres numéros
[ ] Le cours cloné	`ls ~/cours-bi`	`modele-binome`, `pdf`, `prompts`…
[ ] Codex travaille dans Ubuntu	terminal intégré de l'application : `pwd`	`/home/<utilisateur-linux>`
[ ] Compte Free, modèle et effort	sélecteur de modèle ; `/status`	Luna et High, ou ce que votre écran propose, noté
[ ] Compte GitHub et identité Git	`git config --global --list`	nom, adresse noreply, `pull.rebase=false`
[ ] Git relié à GitHub	`gh auth status`	`Logged in to github.com`
[ ] Le binôme se connaît	message	A et B ont le nom d'utilisateur GitHub de l'autre

Ensuite, en séance 1, à deux

Partie D : A crée le dépôt du binôme, invite B et protège `main` ; chacun clone le dépôt et crée son venv. Partie E : la **PR n°1** (`journal-binome`, A auteur, B relecteur). Après la pause, le livret 02 prend le relais avec la PR n°2, dont B sera l'auteur.

01 / DÉMARRER · D · GITHUB À DEUX · SÉANCE 1

Git et GitHub en mots et en images

Git photographie vos fichiers ; GitHub garde l'album partagé. Chaque concept ci-dessous revient à chaque séance.

Dossier local, Git, GitHub

1. Dossier local ~/solen-bi : vos fichiers
2. Git : l'historique caché dans .git
3. GitHub : la copie partagée du binôme

Commit

1. Modifier un fichier
2. `git add` : choisir ce qui part
3. `git commit -m` : photo + message
4. Un point d'historique daté

Branche

1. main : la version validée
2. `git switch -c lecture-csv`
3. Travail isolé sur la branche
4. main reste intacte pendant ce temps

Push et pull

1. Commit sur mon PC
2. `git push` : envoyer à GitHub
3. Le binôme : `git pull`
4. Les deux copies sont alignées

01 / DÉMARRER · D · GITHUB À DEUX · SÉANCE 1

Pull request, revue, merge, CI

Pull request (PR)

1. Branche poussée sur GitHub
2. Compare & pull request
3. Titre + description : quoi, pourquoi, preuve
4. La PR attend une revue

Revue

1. B lit Files changed
2. B relance les contrôles chez lui
3. Approve ou Request changes
4. L'auteur corrige si besoin

Merge

1. PR approuvée + checks verts
2. Merge pull request
3. La branche entre dans main
4. Les deux : git switch main && git pull

CI (intégration continue)

1. Chaque PR déclenche controles.yml
2. Machine Linux neuve chez GitHub
3. Installe et lance les contrôles
4. Coche verte ou croix rouge

Pourquoi tout ce chemin pour deux personnes ?

Parce que c'est ainsi que travaille une équipe BI : personne ne publie seul un chiffre de marge. La PR garde la trace de la demande, du diff, de la preuve et de la décision. À la soutenance, vous montrerez vos PR et vos revues.

01 / DÉMARRER · D · GITHUB À DEUX · SÉANCE 1

GitHub 3 · A crée le dépôt et invite B

A seulement, sur github.com : icône **+** en haut à droite (« Create something new ») → **New repository**. **Owner** : le compte de A. **Repository name** : `solen-<nomA>-<nomB>`. Visibilité **Public** (données 100 % synthétiques). **Ni README, ni .gitignore, ni licence** : le modèle les fournit. **Create repository**.

CAPTURE À RÉALISER · github-nouveau-depot.png

Capture prévue, non encore réalisée : github-nouveau-depot.png. Capture prévue · navigateur, github.com, New repository. Zone : Owner, Repository name, visibilité. Action : Public, aucune case d'initialisation. Réussite : le dépôt vide affiche son adresse HTTPS.

Puis **Settings** (sinon menu **…**) → section **Access** → **Collaborators** → **Add people** → nom d'utilisateur de B → **Add <compteB> to <dépôt>**. **B** accepte l'invitation reçue par e-mail ou notification ; sans cela, B peut lire mais pas pousser.

CAPTURE À RÉALISER · github-inviter.png

Capture prévue, non encore réalisée : github-inviter.png. Capture prévue · navigateur, github.com, Settings → Collaborators. Action : Add people, compte de B. Réussite : B apparaît, invitation en attente.

Erreur courante

B obtient `Permission to <compteA>/<dépôt>.git denied` ou 403 au push : invitation non acceptée, ou B connecté avec un autre compte (`gh auth status`).

01 / DÉMARRER · D · GITHUB À DEUX · SÉANCE 1

GitHub 4 · Cloner et pousser le premier commit

A copie l'adresse HTTPS du dépôt vide, puis dans le terminal Ubuntu :

Terminal Ubuntu (bash)

```
git clone https://github.com/<compteA>/<depot>.git ~/solen-bi
cd ~/solen-bi
cp -r ~/cours-bi/modele-binome/. .
git status
```

Sortie attendue · ne pas recopier

```
warning: You appear to have cloned an empty repository.
Untracked files:  .agents/  .github/  .gitignore  AGENTS.md ...
```

Le clone vide est normal. `/. .` copie **le contenu** du modèle, fichiers cachés compris, dans le dossier courant. Puis :

Terminal Ubuntu (bash)

```
git add .
git commit -m "Départ : modèle du dépôt binôme"
git branch -M main
git push -u origin main
```

Sortie attendue · ne pas recopier

```
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

`git branch -M main` garantit le nom de la branche ; `-u` relie la branche locale à GitHub pour les prochains `git push` et `git pull`. **B**, après avoir accepté l'invitation et après ce push, lance la même première ligne `git clone ... ~/solen-bi`.

Erreurs courantes

`fatal: not a git repository` : `cd ~/solen-bi`. `cp: cannot stat` : le cours n'est pas dans `~/cours-bi` (`ls ~/cours-bi`). `Password authentication is not supported` : refaites GitHub 2.

01 / DÉMARRER · D · GITHUB À DEUX · SÉANCE 1

GitHub 5 · Environnement Python et projet Codex

Chacun, dans son `~/solen-bi`, crée le venv du projet et installe les bibliothèques listées par `requirements.txt`. Le dossier `.venv/` est ignoré par Git (`.gitignore`) : il ne part jamais sur GitHub.

Terminal Ubuntu (bash)

```
cd ~/solen-bi
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
```

Sortie attendue · ne pas recopier

```
(.venv) amira@pc:~/solen-bi$
```

Le préfixe `(.venv)` prouve que le venv est actif. À **chaque nouveau terminal** : `cd ~/solen-bi` puis `source .venv/bin/activate`. Pour sortir : `deactivate`.

Ouvrir le dépôt dans Codex

Mode **Codex** → **Add new project** (Ctrl + O). Windows : `\\wsl$\` → Ubuntu → home → `<utilisateur-linux>` → **solen-bi**. Ubuntu natif : dossier personnel → **solen-bi**. Modèle Luna, effort High.

Demande à Codex · texte à adapter

```
Quelles instructions du projet as-tu lues ? Cite le fichier.
Ne modifie rien.
```

Codex doit citer `AGENTS.md`, le fichier de règles du dépôt qu'il lit avant de travailler.

Erreurs courantes

`ensurepip is not available` : installez `python3-venv` (Windows 1 ou Ubuntu natif 1), puis demandez au professeur avant de recréer le venv. `No such file: requirements.txt` : vous n'êtes pas dans `~/solen-bi`, ou le premier push de A n'est pas encore récupéré (`git pull`).

01 / DÉMARRER · D · GITHUB À DEUX · SÉANCE 1

GitHub 6 · A protège la branche main

Objectif : personne, pas même A, ne pousse directement sur `main` ; tout passe par une PR approuvée par l'autre. **Après** le premier push, **A** : **Settings** → « Code and automation » → **Branches** → **Add classic branch protection rule**.

- **Branch name pattern** : `main`.
- **Require a pull request before merging**, puis **Require approvals** ; **Required number of approvals before merging** : **1**.
- **Do not allow bypassing the above settings** : la règle vaut aussi pour A, propriétaire.
- **Require status checks to pass before merging** : plus tard, après une première CI ; le check s'appelle **Pipeline et contrôles (pull_request)** (le nom contient l'événement qui a lancé la CI).
- **Create**.

CAPTURE À RÉALISER · github-protection-branche.png

Capture prévue, non encore réalisée : github-protection-branche.png. Capture prévue · navigateur, github.com, Settings → Branches du dépôt public. Action : PR requise, 1 approbation, pas de contournement. Réussite : la règle main apparaît dans la liste.

Pourquoi un dépôt public ?

Sur GitHub Free, protection de branche et revue obligatoire ne s'appliquent qu'aux dépôts publics (GitHub Pro les ajoute aux privés) ; les runners Linux standard de GitHub Actions y sont gratuits. Dépôt privé : rien n'est imposé, la règle devient une discipline. GitHub rappelle : « Pull request authors cannot approve their own pull requests. » Si GitHub propose un « ruleset », appelez le professeur.

01 / DÉMARRER · E · PREMIER CYCLE À DEUX · SÉANCE 1

Cycle 1 · Branche, commit, push

Premier cycle, la **PR n°1** du binôme : A présente le binôme dans `notes/JOURNAL.md` ; B relit. La PR n°2 (livret 02, S1-M5) inversera les rôles. Toujours partir d'un `main` à jour, puis ouvrir une branche au nom court et parlant.

Terminal Ubuntu (bash)

```
cd ~/solen-bi
git switch main
git pull
git switch -c journal-binome
```

Sortie attendue · ne pas recopier

```
Switched to a new branch 'journal-binome'
```

Dans Codex, faites la demande de Codex 2 (objectif, fichier, contrôle). Lisez le diff dans l'onglet de revue, puis dans le terminal :

Terminal Ubuntu (bash)

```
git status
git diff
```

`git status` donne la branche et les fichiers changés ; `git diff` les lignes (+ ajoutée, - retirée). Une ligne inattendue : demandez à Codex de l'expliquer ou de l'annuler **avant** de commiter. Ajoutez les fichiers par leur nom : vous savez exactement ce qui part.

Terminal Ubuntu (bash)

```
git add notes/JOURNAL.md
git commit -m "Journal : présentation du binôme"
git push -u origin journal-binome
```

Sortie attendue · ne pas recopier

```
remote: Create a pull request for 'journal-binome' on GitHub by visiting:
remote:   https://github.com/<compteA>/<depot>/pull/new/journal-binome
```

01 / DÉMARRER · E · PREMIER CYCLE À DEUX · SÉANCE 1

Cycle 2 · Ouvrir la pull request

Sur github.com, la page du dépôt affiche **Compare & pull request** pour la branche poussée. Vérifiez **base: main ← compare: journal-binome**.

- Titre : ce que la PR change, en une ligne. Exemple : « Journal : présentation du binôme ».
- Description : **quoi** (fichier touché), **pourquoi**, **preuve** (commande lancée et résultat), **à relire en priorité**.
- À droite, **Reviewers** : choisissez le compte de B.
- **Create pull request**.



Capture prévue · navigateur, github.com, ouverture d'une PR. Zone : base/compare, titre, description, Reviewers. Action : Create pull request. Réussite : la PR porte un numéro (#1) et l'onglet Checks démarre.

Demande à Codex · texte à adapter

```
Rédige la description de PR pour la branche journal-binome :
quoi, pourquoi, preuve, à relire. 6 lignes maximum.
Appuie-toi sur git diff main. Ne modifie aucun fichier.
```

Relisez et corrigez le texte proposé : c'est **votre** PR. Le dépôt contient aussi la méthode `preparer-pr` (dossier `.agents/skills/`) que Codex peut suivre.

01 / DÉMARRER · E · PREMIER CYCLE À DEUX · SÉANCE 1

Cycle 3 · B relit et approuve

B récupère la branche de A et la regarde chez lui, sans rien modifier :

Terminal Ubuntu (bash)

```
cd ~/solen-bi
git fetch
git switch journal-binome
git log --oneline -3
```

Revue Codex en **lecture seule**, puis votre propre lecture :

Demande à Codex · texte à adapter

```
Revue en lecture seule de la branche journal-binome par rapport à
main. Liste au plus 3 remarques : fichier, ligne, risque.
Ne modifie aucun fichier.
```

Sur GitHub, dans la PR : onglet **Files changed** → **Review changes** → écrivez un commentaire → choisissez **Comment**, **Approve** ou **Request changes** → **Submit review**.

CAPTURE À RÉALISER · github-review-approve.png

Capture prévue, non encore réalisée : github-review-approve.png. Capture prévue · navigateur, github.com, PR #1 vue par B. Zone : Files changed → Review changes. Action : Approve avec un commentaire. Réussite : la PR affiche « approved » au nom de B.

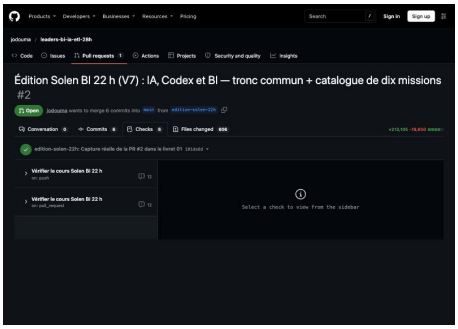
Une revue utile

Une revue cite un fichier et une ligne, dit ce qui est vérifié et ce qui manque. « OK » seul n'est pas une revue. Request changes n'est pas un reproche : c'est le travail du relecteur. L'auteur corrige sur **la même branche**, commit, push : la PR se met à jour.

01 / DÉMARRER · E · PREMIER CYCLE À DEUX · SÉANCE 1

Cycle 4 · Checks, merge, retour à main

En bas de la PR, les checks montrent la CI (`controles.yml`). Coche verte : les contrôles passent sur une machine neuve. En séance 1, la CI vérifie seulement l'installation.



Capture prévue · navigateur, github.com, bas de la PR #1. Réussite : « Pipeline et contrôles (pull_request) » en vert, une approbation, bouton de fusion actif.

PR approuvée et verte : **Merge pull request** → **Confirm merge**, puis le lien pour supprimer la branche sur GitHub. Jamais de fusion sans revue.

CAPTURE À RÉALISER · github-merge.png

Capture prévue, non encore réalisée : github-merge.png. Capture prévue · navigateur, github.com, PR #1. Action : Merge pull request puis Confirm merge. Réussite : bandeau « Merged », main contient la ligne du journal.

Les deux remettent leur copie à jour et effacent la branche locale fusionnée (Git refuse si elle ne l'est pas : aucune perte possible) :

Terminal Ubuntu (bash)

```
git switch main
git pull
git branch -d journal-binome
```

Sortie attendue · ne pas recopier

```
Fast-forward
 notes/JOURNAL.md | 4 ++--
Deleted branch journal-binome (was 9a8b7c6).
```

01 / DÉMARRER · E · PREMIER CYCLE À DEUX · SÉANCE 1

Cycle 4 · Checks, merge, retour à main · suite

Deux « workflows » : même mot, deux choses

Workflow de travail Git/GitHub : la suite de **vos** gestes, branche → commit → push → PR → revue → merge → pull ; vous venez de le faire. **Workflow GitHub Actions** : le fichier `.github/workflows/controles.yml`, que GitHub **exécute tout seul** sur une machine neuve à chaque PR ; on le voit dans l'onglet **Actions** et dans les checks. Le premier se fait à la main, le second tourne sans vous.

01 / DÉMARRER · E · PREMIER CYCLE À DEUX · SÉANCE 1

Erreurs Git courantes : diagnostiquer sans casser

Premier réflexe : `git status`. Il dit le dossier, la branche et l'état des fichiers. Interdits dans ce cours : `git push --force`, `git reset --hard`, supprimer `.git`. Ils effacent du travail.

Symptôme	Cause probable	Réparer sans casser
`fatal: not a git repository`	Mauvais dossier.	`cd ~/solen-bi` puis `git status`.
Commit fait sur main par erreur	Oubli de `git switch -c`.	`git switch -c <branche>` garde le commit sur une nouvelle branche ; poussez-la et ouvrez la PR. Après fusion, `git switch main && git pull` réaligne main.
`Your local changes ... would be overwritten`	Changement non commité avant un `git switch`.	Commitez-le sur la bonne branche, ou `git stash`, changez de branche, puis `git stash pop`.
Push refusé : `protected branch` (GH006)	Push direct sur main.	Normal : créez une branche, poussez-la, ouvrez une PR.
Push refusé : `fetch first` / `non-fast-forward`	Le binôme a poussé sur la même branche.	`git pull` sur cette branche (fusion simple ; si nano s'ouvre : Ctrl + O, Entrée, Ctrl + X), relancez les contrôles, puis `git push`. Jamais `--force`.
PR : `This branch is out-of-date`	main a avancé.	`git fetch`, puis sur votre branche `git merge origin/main`, contrôles, push.
`git pull` : `Need to specify how to reconcile divergent branches`	Réglage `pull.rebase` absent sur ce poste.	`git config --global pull.rebase false` (GitHub 1), puis `git pull` de nouveau.
Check rouge	Contrôle en échec.	**Details** → lisez la fin du journal, relancez la même commande chez vous, corrigez le code sur la branche. Jamais les contrôles ni la référence.
`CONFLICT (content)`	Deux changements sur les mêmes lignes.	Page suivante. Pour annuler la fusion en cours : `git merge --abort`.

01 / DÉMARRER · E · PREMIER CYCLE À DEUX · SÉANCE 1

Résoudre un conflit, calmement

Un conflit n'est pas une panne : Git ne sait pas choisir entre deux versions des mêmes lignes et vous demande de décider. Dans le fichier, il écrit :

Fichier en conflit · extrait

```
<<<<<<< HEAD
- Prochain objectif : compter les lignes des 4 sources.
=======
- Prochain objectif : lire le CSV des ventes.
>>>>>>> origin/main
```

1. Lire les deux versions
2. Demander à Codex d'expliquer, sans modifier
3. Décider à deux, retirer les marqueurs
4. Contrôles, add, commit, push

Demande à Codex · texte à adapter

```
Le fichier notes/JOURNAL.md est en conflit. Explique les deux
versions et leurs conséquences. Ne modifie rien : nous décidons.
```

Après votre décision (Codex peut appliquer le choix que vous lui dictez), vérifiez qu'aucun <<<<<<<, ======= ou >>>>>>> ne reste, puis :

Terminal Ubuntu (bash)

```
git add notes/JOURNAL.md
git commit -m "Fusion de main : conflit du journal résolu"
git push
```

Sur GitHub, si la PR indique un conflit simple, le bouton **Resolve conflicts** ouvre un éditeur : retirez les marqueurs, **Mark as resolved**, puis **Commit merge**. Si le bouton est grisé, résolvez dans le terminal comme ci-dessus.

01 / DÉMARRER · ANNEXES

Annexes · Power BI Desktop et Codex CLI

Power BI Desktop est gratuit. Microsoft propose deux installations ; prenez le **Microsoft Store** : mises à jour automatiques, pas de droits administrateur nécessaires. Il sert en séance 5 : modèle, relations, mesures DAX, filtres, actualisation.

- Microsoft Store → cherchez **Power BI Desktop** → **Installer**.
- Prérequis Microsoft : Windows 10 ou plus récent, au moins 2 Go de RAM disponibles (4 Go conseillés), écran 1440 × 900 ou plus.
- Mise à l'échelle Windows à 100 % (Paramètres → Système → Écran) si certaines fenêtres de Power BI sont coupées.
- Lancez-le une fois : l'écran d'accueil doit s'afficher.

Ubuntu natif

Power BI Desktop ne s'installe que sur Windows. Repérez dès maintenant un poste Windows pour la séance 5 (salle, binôme) ; les exports du pipeline (CSV et manifeste) y seront copiés depuis le dépôt.

Codex CLI (facultatif)

Non requis, non confirmé sur Free

Le cours se fait entièrement dans l'application. La page des tarifs lue le 28/10/2026 cite le CLI à partir du plan Plus ; rien ne confirme qu'il fonctionne avec un compte Free. N'y passez pas de temps en séance.

Pour les curieux, dans Ubuntu (WSL ou natif), l'installeur officiel :

Terminal Ubuntu (bash)

```
curl -fsSL https://chatgpt.com/codex/install.sh | sh
```

Au premier lancement de `codex`, choisissez **Sign in with ChatGPT**. Dans le CLI, `/status` affiche les limites restantes ; l'effort Light de l'application s'y appelle Low.

Si le CLI refuse votre compte

C'est l'état attendu sur Free : revenez à l'application. Ne payez rien pour ce cours et ne créez pas de clé API.

01 / DÉMARRER · ANNEXES

Récapitulatif : prêt pour la suite de la séance 1

À ce stade, je sais…

- …dire où vivent mes dossiers (`~/cours-bi`, `~/solen-bi`) et les ouvrir dans Codex.
- …régler Codex : Luna, effort High, et lire mon usage avec `/status`.
- …expliquer en mots : dossier local, Git, GitHub, commit, branche, push/pull, PR, revue, merge, CI.
- …faire un cycle complet : branche, demande, diff, commit, push, PR, revue, merge, retour à main.
- …réagir à un push refusé, une branche en retard, un check rouge ou un conflit sans `--force` ni `reset --hard`.

Preuve observable du binôme

Sur GitHub : un dépôt public `solen-<nomA>-<nomB>`, B collaborateur, main protégée, la PR n°1 (`journal-binome`, A auteur) fusionnée avec une revue écrite de B. Chez chacun : `git status` sur main propre, `(.venv)` actif, `gh auth status` connecté.

La suite

Après la pause, ouvrez le livret 02 (tronc commun, mission Solen Commerce 360) : la PR n°2 de la séance (`cadrage-binome`) aura **B** pour auteur et A pour relecteur. Gardez l'aide-mémoire Git · Codex · BI ouvert pendant les séances.

D'où viennent les étapes ?

Documentation officielle consultée le 28 octobre 2026 : OpenAI (tarifs, modèles, application Windows et Linux, WSL, CLI, skills, AGENTS.md), Microsoft Learn (WSL, Power BI Desktop), Ubuntu, Python, Git et GitHub Docs. Liste complète : `docs/setup/sources.json`. Ces lectures ne sont pas un essai réel sur votre poste ni sur un compte Free ; les captures sont réalisées séparément et légendées.